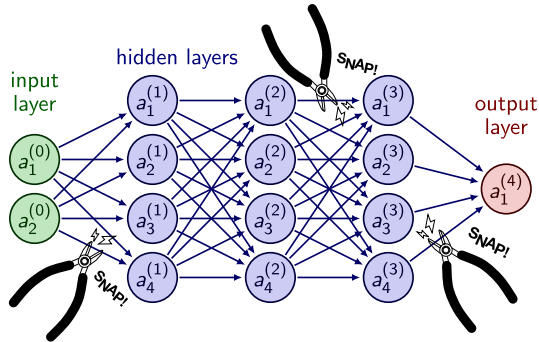


Structure and Interpretation of Deep Neural Networks

Pavanakumar Mohanamuraly



हिरण्मये॑ न पात्रे॑ण सत्यस्यापि॑हितं मुखम् ।

तत्त्वं॑ पू॒षन्नपावृ॑णु सत्यध॑र्माय दृष्टये॑ ॥ १५ ॥

The door of the Truth is covered with a golden lid. Remove, O Sun (Nourisher), the covering, that I, the worshipper of the Truth may behold it.

ॐ शान्तिः॑ शान्तिः॑ शान्तिः॑ ॥

Deep Learning

- ▶ Deep learning is a specific kind of machine learning
- ▶ Machine learning algorithm is an algorithm that is able to learn from data
- ▶ *A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .*¹
- ▶ Task T : Classification, Regression, Denoising, etc.
- ▶ Performance measure P : Measure of accuracy or performance of task T using the model
- ▶ Experience E : Dataset or test set used for training
 - ▶ Supervised learning : Labelled data
 - ▶ Unsupervised learning : Raw data (denoising, clustering)

¹Mitchell, T. M. (1997). Machine Learning. McGraw-Hill, New York

Deep Neural Network

$$\mathbf{x}_{out} = g(\mathbf{x}_{in}) = \sigma(\mathbf{b} - \mathbf{A}\mathbf{x}_{in})$$

Efficient gradients from
back-propagation (AD)

$$\mathbf{A} \leftarrow \mathbf{A} + \left(\frac{\partial g}{\partial \mathbf{A}} \right)^T \mathbf{x}_{out}$$

$$\mathbf{b} \leftarrow \mathbf{b} + \left(\frac{\partial g}{\partial \mathbf{b}} \right)^T \mathbf{x}_{out}$$

Deep combinatorics produces
rich representations

Parameters optimised using
gradient-based methods



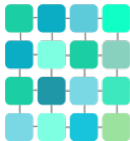
TEMPORAL/
SEQUENCE



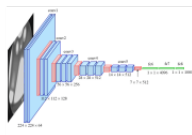
Have a nice day! :)



Recurrent neural
network (RNN)



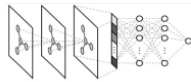
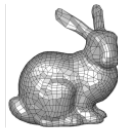
REGULAR
TOPOLOGY



Convolutional neural
network (CNN)



RELATIONAL DATA
AND UNSTRUCTURED
TOPOLOGY



Graph network
(GNN/GCN)

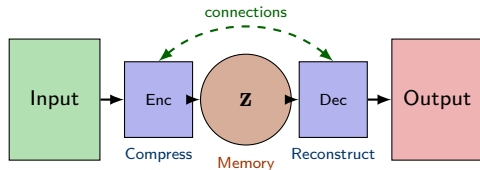
Encoders: Memory Units of Deep Networks

Key Insight: Deep networks can be viewed as:

- ▶ **Encoder units** – store compressed representations (memory)
- ▶ **Connecting units** – transform and evolve these representations toward solutions

The *efficacy* of a network depends on:

1. Quality of encoded memories
2. Structure of connections between encoders



Memory z captures essential features; connections govern information flow

Autoencoders: Learning Efficient Representations

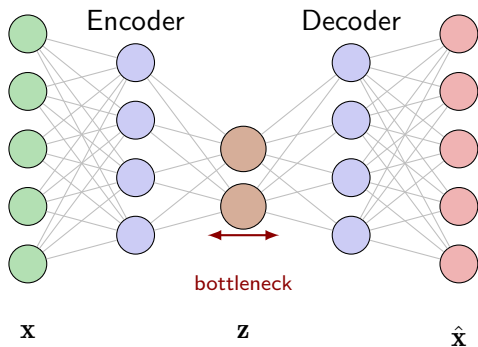
Encoder: $\mathbf{z} = f_{\theta}(\mathbf{x})$

Decoder: $\hat{\mathbf{x}} = g_{\phi}(\mathbf{z})$

Loss: $\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$

Key Properties:

- ▶ *Bottleneck* forces compression
 $\dim(\mathbf{z}) \ll \dim(\mathbf{x})$
- ▶ Learns *latent features* without supervision
- ▶ Variants: VAE, Sparse AE, Denoising AE



Applications: Dimensionality reduction, anomaly detection, generative models, feature learning for downstream tasks

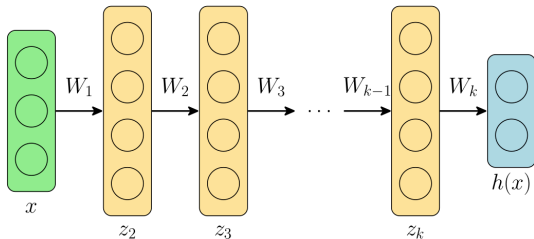
Deep Feed-forward Network

Typical k -layer deep FF network

$$\mathbf{z}_1 = \mathbf{x}$$

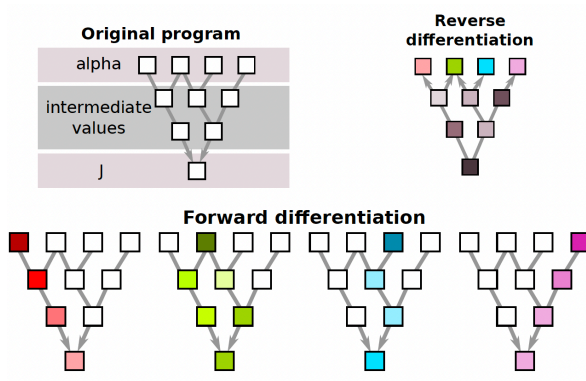
$$\mathbf{z}_{i+1} = \sigma(\mathbf{W}_i \mathbf{z}_i + \mathbf{b}_i), \quad i=1, \dots, k-1$$

$$\mathbf{h}(\mathbf{x}) = \mathbf{W}_k \mathbf{z}_k + \mathbf{b}_k$$



- ▶ $\mathbf{x}$ are the input features
- ▶ $\mathbf{z}_i$ are called hidden layers (intermediate outputs)
- ▶ $\mathbf{h}$ are the output features
- ▶ k is the depth of the feedforward network

Problem of Gradient Checkpointing



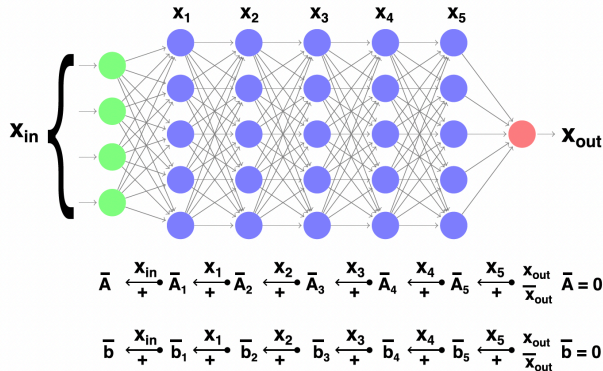
Reverse AD needs to restore memory states in the reverse order of their normal appearance

Problem of Gradient Checkpointing

$$g(\mathbf{x}_{in}) := \mathbf{x}_{out} = \sigma(\mathbf{A}\mathbf{x}_{in} + \mathbf{b})$$

$$\begin{aligned}\bar{\mathbf{A}} &\leftarrow \bar{\mathbf{A}} + \left(\frac{\partial g}{\partial \mathbf{A}}\right)^T \bar{\mathbf{x}}_{out} \\ &= \bar{\mathbf{A}} + \left(\frac{\partial R}{\partial \mathbf{A}}\right)^T \left(\frac{\partial \sigma}{\partial R}\right)^T \bar{\mathbf{x}}_{out}\end{aligned}$$

$$\begin{aligned}\bar{\mathbf{b}} &\leftarrow \bar{\mathbf{b}} + \left(\frac{\partial g}{\partial \mathbf{b}}\right)^T \bar{\mathbf{x}}_{out} \\ &= \bar{\mathbf{b}} + \left(\frac{\partial R}{\partial \mathbf{b}}\right)^T \left(\frac{\partial \sigma}{\partial R}\right)^T \bar{\mathbf{x}}_{out}\end{aligned}$$

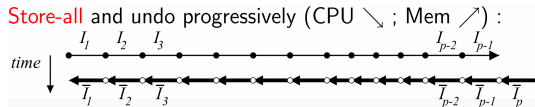


- Unsteady CFD adjoints-gradient
- Deep neural networks

Checkpointing Strategy : The Two Extremes

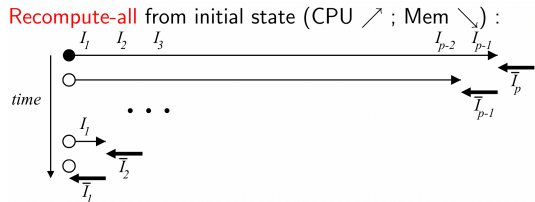
Store All:

- ▶ Memory $\propto T$ (tape length)
- ▶ Time: $1 \times$ forward + $1 \times$ reverse
- ▶ Prohibitive for large T !



Recompute All:

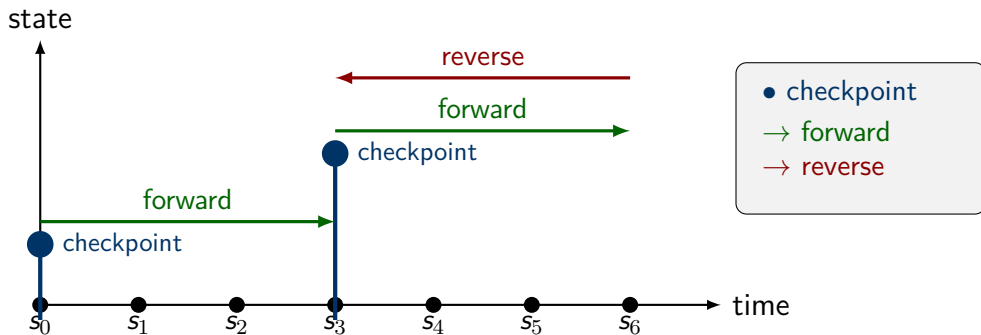
- ▶ Memory $\propto R$ (state size)
- ▶ Time: $O(n^2)$ – restart from s_0 each time
- ▶ Prohibitive runtime!



Can we do better? Yes! – The REVOLVE Algorithm (Griewank, 1992)

REVOLVE: Optimal Checkpointing Strategy

Key Idea: Use d snapshots (checkpoints) and allow t extra forward sweeps



- ▶ Save state at strategic checkpoints, recompute segments as needed
- ▶ Trade-off: More checkpoints $\Rightarrow$ less recomputation (but more memory)
- ▶ **Question:** What is the *optimal* checkpoint placement?

Griewank's Key Result : The Binomial Solution

With d snapshots and t extra forward sweeps, the *maximum* number of steps η that can be reversed is:

$$\eta(d, t) = \binom{d+t}{t} = \frac{(d+t)!}{d!t!}$$

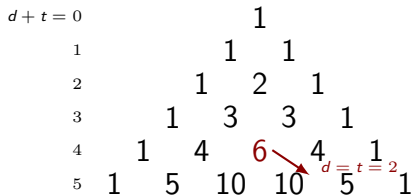
Examples:

- ▶ $d = 3, t = 5 \Rightarrow \eta = \binom{8}{5} = 56$ steps
- ▶ $d = t = 12 \Rightarrow \eta \approx 10^6$ steps!

Complexity grows logarithmically:

$$d = t \approx \log_4(\eta)$$

Pascal's Triangle Structure



$$\eta(d, t) = \eta(d-1, t) + \eta(d, t-1)$$

(Each entry = sum of two above)

Treeverse Recursive Algorithm

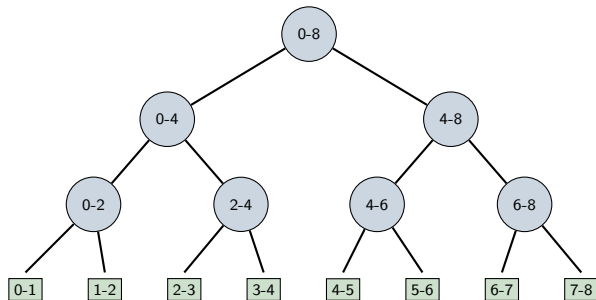
1. Take snapshot
2. Advance to midpoint
3. Reverse second half
4. Restore snapshot
5. Reverse first half

Optimal partition:

$$\kappa = \left\lceil \frac{\delta \cdot \sigma + \tau \cdot \phi}{\tau + \delta} \right\rceil$$

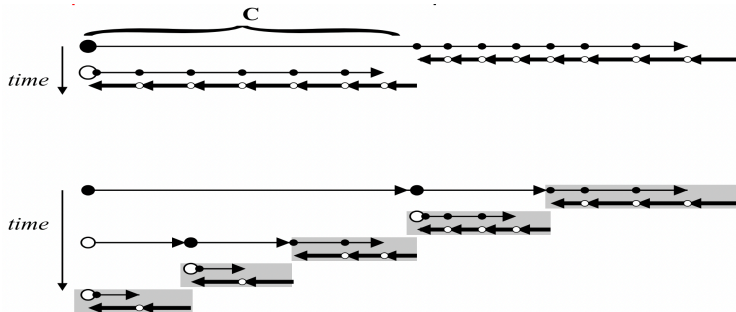
where, δ =remaining snapshots, τ =remaining sweeps

Result: Memory $\sim O(d \cdot R)$, Time $\sim O((1 + t) \cdot W)$ where $d + t \sim \log(\eta)$



Recursive call tree

Binary Checkpointing Summary



- ▶ Use d snapshots and allow t extra forward sweeps
- ▶ **REVOLVE** (Griewank): optimal binomial partitioning $\eta(d, t) = \binom{d+t}{t}$
- ▶ Memory and CPU overhead grow like $\mathbf{O}(\log(\eta))$ – *logarithmic!*
- ▶ Enables adjoint computation for virtually **any problem size**

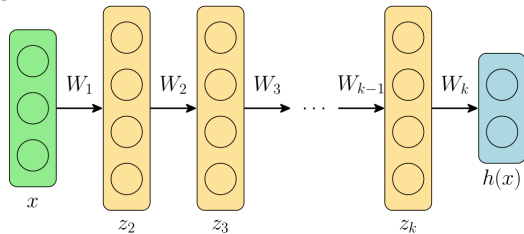
Connection with implicit functions*

Typical k -layer deep network

$$\mathbf{z}_1 = \mathbf{x}$$

$$\mathbf{z}_{i+1} = \sigma(\mathbf{W}_i \mathbf{z}_i + \mathbf{b}_i), \quad i=1, \dots, k-1$$

$$\mathbf{h}(\mathbf{x}) = \mathbf{W}_k \mathbf{z}_k + \mathbf{b}_k$$

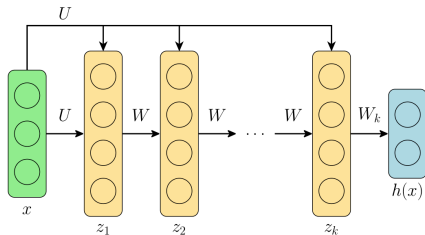


Consider same weight/bias

$$\mathbf{z}_1 = \mathbf{0}$$

$$\mathbf{z}_{i+1} = \sigma(\mathbf{W} \mathbf{z}_i + \mathbf{U} \mathbf{x} + \mathbf{b})$$

$$\mathbf{h}(\mathbf{x}) = \mathbf{W}_k \mathbf{z}_k + \mathbf{b}_k$$



For ∞ layers with same weights we get a non-linear fixed-point

$$\mathbf{z}^* = \sigma(\mathbf{W} \mathbf{z}^* + \mathbf{U} \mathbf{x} + \mathbf{b}) \quad \text{or} \quad \mathbf{z} = f(\mathbf{z}, \mathbf{a})$$

* *Deep Equilibrium Models*, Bai et. al., NeurIPS 2019.

Implicit Functions and Attractive Fixed-points

- ▶ Christianson² in his seminal work shows that a fixed-point of the form

$$z := \Phi(z, x) \quad (1)$$

- ▶ Φ is well behaved iterative construction that converges to the value z_* , the adjoint-gradient only requires values from the last iteration

$$y := f(z_*, x) \quad (2)$$

- ▶ In BWD sweep, adjoint loop solves the adjoint FP equation

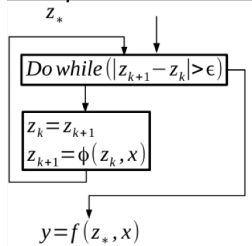
$$\bar{w} := \bar{w} \cdot \frac{\partial}{\partial z} \Phi(z_*, x) + \bar{z} \quad (3)$$

- ▶ $\bar{w}$ is returned by the adjoint of downstream computation of f

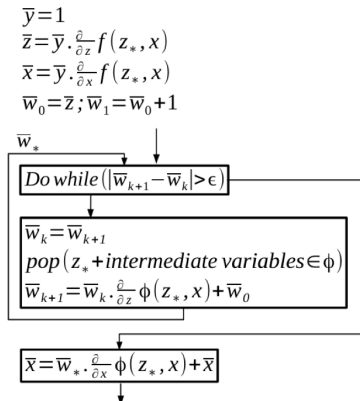
²Reverse accumulation and attractive fixed points, OMS, vol. 3(4), 1992.

Implicit Functions and Attractive Fixed-points³

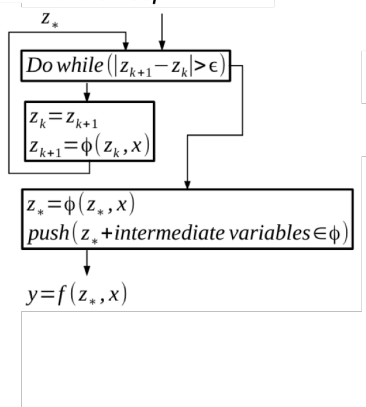
Fixed-point iteration



Backward Sweep



Forward Sweep



³Implementation and measurements of an efficient Fixed Point Adjoint, A Taftaf, et. al., EUROGEN 2015

Deep combinatorics?

Strong connect between ODEs and Neural Networks⁴

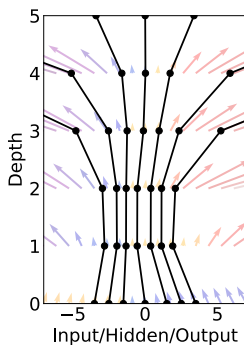
ResNet/RNN are sequence of transformations on hidden state $\mathbf{h}$,

$$\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t)$$

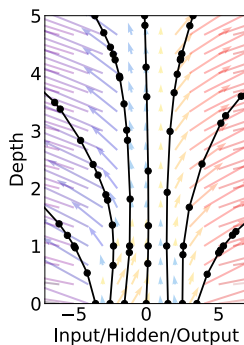
In the limit of ∞ layers

$$\frac{d\mathbf{h}}{dt} = f(\mathbf{h}, t, \theta)$$

Residual Network



ODE Network



⁴Neural Ordinary Differential Equations, Chen et. al., NeurIPS 2018.

Deep Network Training (Vanishing Gradients)

Chain Rule in Deep Networks: For n -layer network, gradient at layer 1:

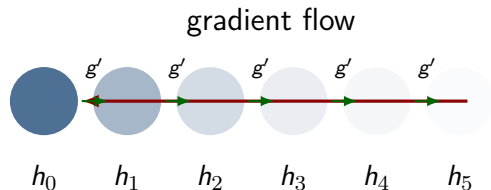
$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial h_n} \cdot \underbrace{g'_{n-1} \cdot g'_{n-2} \cdot \dots \cdot g'_1}_{n-1 \text{ terms}} \cdot x_0$$

Analytical Example:

Let $g'_i = \sigma'(\cdot)$ for sigmoid/tanh where $|g'_i| \leq 0.25$

Layers	Gradient Scale
$n = 5$	$(0.25)^4 \approx 0.004$
$n = 10$	$(0.25)^9 \approx 10^{-6}$
$n = 20$	$(0.25)^{19} \approx 10^{-12}$
$n = 50$	$(0.25)^{49} \approx 10^{-30}$

Gradients vanish exponentially!



Circle opacity $\propto$ gradient magnitude

Early layers receive near-zero gradients

Exploding Gradients: The Other Extreme

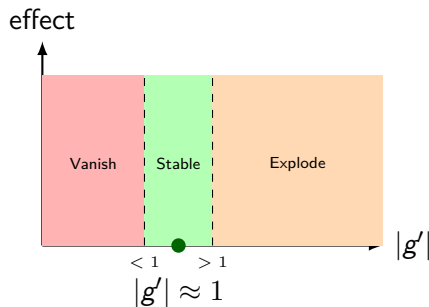
When $|g'_i| > 1$: Gradients grow exponentially! **Analytical Example:**

Let $g'_i = 2$ (e.g., poorly initialized ReLU)

Layers	Gradient Scale
$n = 5$	$2^4 = 16$
$n = 10$	$2^9 = 512$
$n = 20$	$2^{19} \approx 5 \times 10^5$
$n = 50$	$2^{49} \approx 5 \times 10^{14}$

Causes NaN, training divergence!

The Goldilocks Zone:



Goal: Keep $\prod_i |g'_i| \approx 1$

Solution: Careful weight initialization (Xavier/He) + architecture design (ResNets)

Xavier⁵ Initialisation - variance constant across layers

For layer l : $y^{(l)} = W^{(l)} x^{(l-1)}$

Variance propagation:

$$\text{Var}(y^{(l)}) = n^{(l-1)} \cdot \text{Var}(W^{(l)}) \cdot \text{Var}(x^{(l-1)})$$

For $\text{Var}(y^{(l)}) = \text{Var}(x^{(l-1)})$, we need:

$$\text{Var}(W^{(l)}) = \frac{1}{n^{(l-1)}}$$

Xavier (Glorot) Init:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

He Init (for ReLU):

ReLU zeros out half the values, so:

$$\text{Var}(W^{(l)}) = \frac{2}{n^{(l-1)}}$$

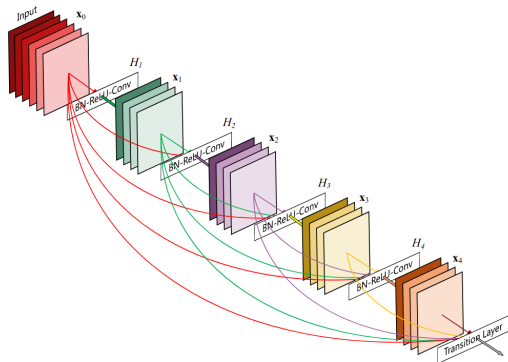
Practical Rules:

- ▶ **tanh/sigmoid:** Xavier
- ▶ **ReLU/Leaky ReLU:** He
- ▶ **SELU:** LeCun init

Result: Gradients stay $\approx O(1)$ across all layers!

⁵ Understanding the difficulty of training deep feedforward neural networks, PMLR 9:249-256, 2010.

Skip Connections to the Rescue^{6 7}



- ▶ Skip connections one of the important innovations in DL
- ▶ Provide direct gradient paths, allowing gradients to flow more easily to earlier layers
- ▶ Makes optimization landscape smoother by allowing gradients to flow more easily during training
- ▶ DenseNet, ResNet, UNets, Transformers employ them

⁶ *The Shattered Gradients Problem: If ResNets are the Answer, Then What is the Question?*, Veit et al., NIPS 2016.

⁷ *Attention Is All You Need*, A Vaswani, et. al., 2017

Similarities with Numerical ODE Solution

- ▶ Build on insights from DEQ and Neural ODEs
- ▶ Use traditional ODE methods to discover/impose structure?
- ▶ Emphasis on structure rather than plain depth (inter-connect)
- ▶ Connect depth to sub-step of multi-step Runge-Kutta methods
- ▶ RK most generic multi-step time integration formulation
- ▶ Show possible ways to understand and exploit structure in NN

Runge-Kutta methods and numerical solution of ODEs

$$y^{n+1} = y^n + h \sum_{i=1}^s b_i k_i$$

$$k_i = f\left(t^n + c_i h, y^n + h \sum_{j=1}^s a_{ij} k_j\right)$$

Butcher's tableau for s -stage Runge-Kutta methods

c_1	a_{11}	a_{12}	$\dots$	$a_{1,s}$
c_2	a_{21}	a_{22}	$\dots$	$a_{2,s}$
c_3	a_{31}	a_{32}	$\dots$	$a_{3,s}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
c_s	$a_{s,1}$	$a_{s,2}$	$\dots$	$a_{s,s}$
	b_1	b_2	$\dots$	b_s

(a) *Implicit method*

0	0	$\dots$	$\dots$	$\dots$	0
c_2	a_{21}	0	$\dots$	$\dots$	0
c_3	a_{31}	a_{32}	0	$\dots$	0
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\ddots$	$\vdots$
c_s	$a_{s,1}$	$a_{s,2}$	$\dots$	$a_{s,s-1}$	0
	b_1	b_2	$\dots$	b_{s-1}	b_s

(b) *Explicit method*

Data-flow graph of 3-stage Runge-Kutta method

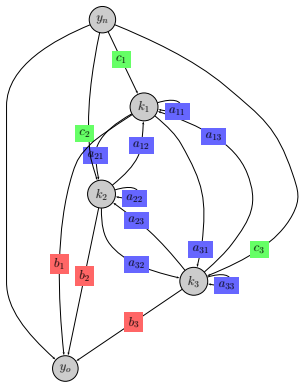


Figure: Implicit Runge-Kutta

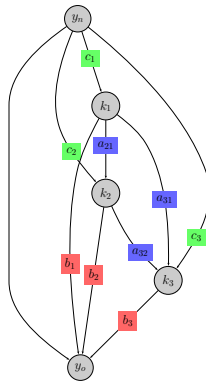
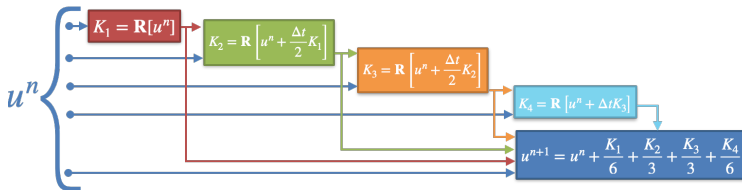


Figure: Explicit Runge-Kutta

RK4 family of schemes

- ▶ RK4 butcher tableau has multiple solutions
- ▶ Discover schemes using optimisation imposing objective(s)
 - ▶ Low dissipation/dispersion (LSDIRK in aero-acoustics)
 - ▶ Stable low-storage versions (Jameson steady-state CFD)
 - ▶ Implicit versions for stiff ODE type problems (chemistry/reaction)



Guaranteed error improvement

- ▶ Convergence property of Taylor series guarantees improvement in error with better sampling i.e, as $\Delta t \rightarrow 0$ then $\epsilon \rightarrow 0$
- ▶ Cannot guarantee this in general for a Neural Network
- ▶ Structural issue? If so can we mitigate by providing the right structure (based on RK network analysis)?

Come ... let us find out !

ॐ (३) क्रतो॑ स्मर॑ कृत॑ स्मर॑ क्रतो॑ स्मर॑ कृत॑ स्मर॑ ॥ १७ ॥

O my mind, remember all that has been done. Remember – remember all that has been done.

ॐ शा॒न्तिः शा॒न्तिः शा॒न्तिः ॥